

## Модельовані структури даних Python. Черга



**Задача.** Побудувати програмну модель організації прикордонного митного контролю. Використати структуру даних черги.

Подібні задачі виникають в різноманітних сервісах, пов'язаних з організацією прикордонних служб.

Задачу моделювати для вантажних перевезень. Вантажні авто прибувають до кордону, стають в чергу на митний огляд, проходять огляд, після огляду рухаються далі до пункту призначення або отримують інші розпорядження.

*Створити власний клас черги:*

- 1) з методами **базовими** для черги як структури;
- 2) з методами **зв'язаними** зі змістом задачі;
- 3) з засобами **контролю і захисту** (питання стійкості і безпеки).

Частина 1. Побудувати шаблон митної декларації. Митна декларація повинна мати всі необхідні відомості про перевезення товару. Шаблон будувати на основі структури словника. Для прикладу, шаблон відомостей може мати такі ключі і значення (на змістовому рівні):

```
avtonn = { авто : марка, номер_авто : номер, власник_авто : компанія,
            дата : рік-місяць-день,
            звідки : (країна, місто), куди : (країна, місто),
            товари : [ перелік_товарів - (назва, вартість) ],
            договір_перевезення : є / нема,
            інші_дані : . . . . .
          }
```

Можна вважати для спрощення задачі, що кожне авто перевозить лише один вид товару. Точний зміст шаблону визначити самостійно.

Частина 2. Побудувати окремо список митних декларацій авто, які мають прибути до кордону (є в дорозі) і пройти митний контроль. Список будувати в довільному порядку і використати умовні дані (але правдоподібні). Для будови списку можна використати шаблон і кожне авто заповнити вручну. Можна дещо автоматизувати, врахувавши, що шаблон завжди буде мати однаковий перелік ключів, а змінними будуть лише значення. Наприклад:

```
listkey = [ авто, номер_авто, власник_авто, дата, . . . ]
           # список містить перелік ключів шаблону
data1 = [ "Volvo", D1093AH, "Green day company", 2020.10.12,
           ("Poland", "Wroclaw"), ("Україна", "Вінниця"), . . . ]
avto1 = dict ( zip ( listkey, data1 ) ) # декларація для одного авто
avto2 = . . . . .
```

**Частина 3.** Будемо використовувати дві черги: загальна, зелений коридор. З кожною чергою можна виконати операції відповідно до функцій програмного класу черги.

**Частина 4.** Операції митного контролю. [ *Методи, зв'язані зі змістом задачі* ]

- ❖ поставити авто в чергу для огляду; (одну або другу)
- ❖ закінчити огляд і дозволити перетин кордону – читати з черги;
- ❖ перевести авто з однієї черги в кінець другої;
- ❖ викреслити авто з черги (немає дозволу на перетин кордону);
- ❖ скласти перелік авто (місце в черзі), які везуть вказаний товар;
- ❖ яке авто має товар найбільшої вартості?
- ❖ таблиця товарів і цін, перевезених через кордон (підсумок);
- ❖ які авто прямують до Одеси?
- ❖ інші операції – додати самостійно ще 3-4 операції; наприклад – контроль перевезення наркотичних чи вибухових речовин:



[Про мене](#)



Який програмно модельований алгоритм можна пропонувати для такої операції? Що було б корисно додати до шаблону митної декларації, щоб уможливити алгоритмічне моделювання цієї операції хоча б з певною достовірністю:

<https://uk.wikipedia.org/wiki/Достовірність>

**Частина 5.** Диспетчер пункту прикордонного контролю. Скласти довільний сценарій комбінації перелічених операцій митного контролю (частини 4) і відповідну програму. Авто вибирати з переліку частини 2. Друкувати необхідні результати операцій в протоколі.

*Важливо!* Сценарій має передбачати в тому числі спроби некоректних операцій. Відповідно в протоколі має бути відображено реагування методів контролю і захисту на такі спроби.

---

## Вимоги до реалізації

1) Завдання реалізувати в цілому за технологією ООП. Весь програмний код оформити одним модулем (файлом). Ознайомитись з питаннями якості програмного забезпечення і застосувати до цієї задачі в доцільному переліку:

[https://uk.wikipedia.org/wiki/Якість\\_програмного\\_забезпечення](https://uk.wikipedia.org/wiki/Якість_програмного_забезпечення)

2) Реалізувати власний клас черги, зовнішній, доступний для всіх функцій, побудований з методами *базовими, зв'язаними і засобами контролю і захисту*. На його основі виконати програмування завдань.

3) Кожен метод частини 4 завдання коментувати за текстом програми, на забути пояснити призначення параметрів методів.

- 4) Побудувати і пояснити *коментарями до програмного коду* сценарій, визначений як частина 5 завдання. Показати в своєму сценарії приклади використання *кожного* методу частини 4 завдання. Надрукувати *протокол сценарію* в окремий текстовий файл. Очевидно, що для цього треба додати до функцій програми оператори друкування протокольних повідомлень, які би включали результати кожного кроку обов'язково з коментарями.
- 5) Окремо зробити скріншоти виконання на своєму комп'ютері і додати до протоколу виконання, або записати в окремий файл з *підписами* кожного скріншота.
- 6) Щоб проєкт мав належний рівень, список митних декларацій (частина 2) має бути в обсязі 15-20 елементів або більше.
- 

7) [ Факультативно ] З навчальною метою можна використати як допоміжний клас даних модуля `dataclasses` для частини 1 і частини 2 завдання:

<https://docs.python.org/3.10/library/dataclasses.html?highlight=dataclasses#module-dataclasses>  
<https://realpython.com/python-data-classes/>

В результаті надіслати у відповідь такі файли:

- файл програмного коду;
- файл класу черги – якщо клас визначений окремо;
- файл з списком митних декларацій – якщо визначено окремо;
- текстовий файл протоколу виконання – обов'язково для підтвердження;
- приклади скріншотів виконання (або разом з протоколом).

|                                                       |
|-------------------------------------------------------|
| Файли надіслати окремими вкладеннями, НЕ АРХІВУВАТИ ! |
|-------------------------------------------------------|